

Task 8: Employee Management Module using Spring Core

Design a simple Employee Management module using Spring Core. Demonstrate Inversion of Control and Dependency Injection using annotations such as `@Component` and `@Autowired`. Use BeanFactory to manage beans and store employee data in memory.

AIM:

To create a simple Employee Management module using Spring Core that demonstrates Inversion of Control (IoC) and Dependency Injection (DI) using annotations such as `@Component` and `@Autowired`, use BeanFactory (via ApplicationContext) to manage beans, and store employee data in memory using a HashMap.

ALGORITHM:

1. Start
2. Create a Maven project with Spring Boot web dependency.
3. Define an Employee model class with fields: id, name, department, salary.
4. Create an EmployeeDAO interface with CRUD methods: `addEmployee()`, `getEmployeeById()`, `getAllEmployees()`, `updateEmployee()`, `deleteEmployee()`.
5. Implement EmployeeDAO in EmployeeDAOImpl using a HashMap for in-memory storage. Annotate with `@Component`.
6. Create EmployeeService class with `@Component`. Inject EmployeeDAO using `@Autowired` (Dependency Injection).
7. Create EmployeeController with `@RestController`. Inject EmployeeService using `@Autowired`. Define REST endpoints for CRUD.
8. Create MainApp with `@SpringBootApplication`. Spring container (BeanFactory) manages all beans (Inversion of Control).
9. Create index.html web UI to interact with the REST API.
10. Run the application on localhost:8080.
11. Perform CRUD operations: Add, View, Update, Delete employees.
12. Display results on console and browser.
13. Stop

PROCEDURE:

Step 1: Create a new Maven project in IntelliJ IDEA with Group ID com.employee and Artifact ID EmployeeManagement.

Step 2: Add Spring Boot spring-boot-starter-web dependency in pom.xml to enable Spring Core and web support.

Step 3: Create an Employee model class with fields: id, name, department, salary along with constructors, getters, setters, and toString().

Step 4: Create an EmployeeDAO interface declaring CRUD methods: addEmployee(), getEmployeeById(), getAllEmployees(), updateEmployee(), and deleteEmployee().

Step 5: Implement the EmployeeDAO interface in EmployeeDAOImpl class using a HashMap for in-memory storage. Annotate it with @Component to register it as a Spring-managed bean.

Step 6: Create an EmployeeService class annotated with @Component. Use @Autowired to inject EmployeeDAO — demonstrating Dependency Injection.

Step 7: Create an EmployeeController class annotated with @RestController. Use @Autowired to inject EmployeeService and expose REST API endpoints for CRUD operations.

Step 8: Create MainApp.java with @SpringBootApplication annotation. The Spring container (BeanFactory/Application Context) automatically manages all bean lifecycles — demonstrating Inversion of Control.

Step 9: Create an index.html file inside src/main/resources/static/ to provide a web-based UI for the Employee Management system.

Step 10: Run the application and access it at <http://localhost:8080>. Perform Add, View, Update, and Delete operations on employees.

PROGRAM:

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.5</version>
  </parent>
  <groupId>com.employee</groupId>
  <artifactId>EmployeeManagement</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

2. Employee.java (Model)

```
package com.employee.model;
public class Employee {
  private int id;
  private String name;
  private String department;
  private double salary;
  public Employee() {}
  public Employee(int id, String name, String department, double salary) {
    this.id = id;
    this.name = name;
    this.department = department;
  }
}
```

```

        this.salary = salary;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
    @Override
    public String toString() {
        return "Employee [ID=" + id + ", Name=" + name +
            ", Department=" + department + ", Salary=" + salary + "]";
    }
}

```

3. EmployeeDAO.java (Interface)

```

package com.employee.dao;
import com.employee.model.Employee;
import java.util.List;
public interface EmployeeDAO {
    void addEmployee(Employee employee);
    Employee getEmployeeById(int id);
    List<Employee> getAllEmployees();
    void updateEmployee(Employee employee);
    void deleteEmployee(int id);
}

```

4. EmployeeDAOImpl.java (@Component — In-Memory Storage)

```

package com.employee.dao;
import com.employee.model.Employee;
import org.springframework.stereotype.Component;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
@Component
public class EmployeeDAOImpl implements EmployeeDAO {
    private Map<Integer, Employee> employeeMap = new HashMap<>();
    @Override
    public void addEmployee(Employee employee) {
        employeeMap.put(employee.getId(), employee);
        System.out.println("Employee added: " + employee.getName());
    }
    @Override
    public Employee getEmployeeById(int id) {
        return employeeMap.get(id);
    }
}

```

```

@Override
public List<Employee> getAllEmployees() {
    return new ArrayList<>(employeeMap.values());
}
@Override
public void updateEmployee(Employee employee) {
    if (employeeMap.containsKey(employee.getId())) {
        employeeMap.put(employee.getId(), employee);
        System.out.println("Employee updated: " + employee.getName());
    } else {
        System.out.println("Employee not found with ID: " + employee.getId());
    }
}
@Override
public void deleteEmployee(int id) {
    Employee removed = employeeMap.remove(id);
    if (removed != null) {
        System.out.println("Employee deleted: " + removed.getName());
    } else {
        System.out.println("Employee not found with ID: " + id);
    }
}
}

```

5. EmployeeService.java (@Component + @Autowired)

```

package com.employee.service;
import com.employee.dao.EmployeeDAO;
import com.employee.model.Employee;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import java.util.List;
@Component
public class EmployeeService {
    @Autowired
    private EmployeeDAO employeeDAO;
    public void addEmployee(Employee employee) {
        employeeDAO.addEmployee(employee);
    }
    public Employee getEmployeeById(int id) {
        return employeeDAO.getEmployeeById(id);
    }
    public List<Employee> getAllEmployees() {
        return employeeDAO.getAllEmployees();
    }
    public void updateEmployee(Employee employee) {
        employeeDAO.updateEmployee(employee);
    }
    public void deleteEmployee(int id) {
        employeeDAO.deleteEmployee(id);
    }
}

```

```
}
```

6. EmployeeController.java (@RestController + @Autowired)

```
package com.employee.controller;
import com.employee.model.Employee;
import com.employee.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/employees")
@CrossOrigin(origins = "*")
public class EmployeeController {
    @Autowired
    private EmployeeService employeeService;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeService.getAllEmployees();
    }

    @GetMapping("/{id}")
    public Employee getEmployeeById(@PathVariable int id) {
        return employeeService.getEmployeeById(id);
    }

    @PostMapping
    public String addEmployee(@RequestBody Employee employee) {
        employeeService.addEmployee(employee);
        return "Employee added: " + employee.getName();
    }

    @PutMapping("/{id}")
    public String updateEmployee(@PathVariable int id, @RequestBody Employee employee) {
        employee.setId(id);
        employeeService.updateEmployee(employee);
        return "Employee updated: " + employee.getName();
    }

    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable int id) {
        employeeService.deleteEmployee(id);
        return "Employee deleted with ID: " + id;
    }
}
```

7. MainApp.java (Spring Boot Application — IoC via BeanFactory)

```
package com.employee.main;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;

@SpringBootApplication
@ComponentScan(basePackages = "com.employee")
public class MainApp {
```

```

public static void main(String[] args) {
    SpringApplication.run(MainApp.class, args);
    System.out.println("\n===== Employee Management System Started =====");
    System.out.println("Open browser: http://localhost:8080");
    System.out.println("=====\\n");
}
}

```

8. application.properties

```

server.port=8080
spring.application.name=EmployeeManagement

```

9. index.html (Web UI — src/main/resources/static/index.html)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Employee Management System</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; background: #eef2f7; display: flex; justify-content: center;
padding: 30px; }
    .container { background: #fff; padding: 25px; width: 700px; border-radius: 10px; box-shadow: 0
5px 15px rgba(0,0,0,0.2); }
    h2 { text-align: center; margin-bottom: 15px; color: #333; }
    .form-group { margin-bottom: 10px; }
    .form-group label { display: block; font-weight: bold; font-size: 14px; margin-bottom: 4px; }
    input, button { width: 100%; padding: 8px; margin: 4px 0; font-size: 14px; }
    input { border: 1px solid #ccc; border-radius: 5px; }
    input:focus { outline: none; border-color: #1a73e8; }
    .btn-row { display: flex; gap: 10px; margin-top: 10px; }
    button { border: none; border-radius: 5px; cursor: pointer; font-weight: bold; padding: 10px; }
    .btn-add { background: #1a73e8; color: white; }
    .btn-add:hover { background: #1557b0; }
    .btn-clear { background: #ddd; color: #333; }
    .btn-clear:hover { background: #ccc; }
    .btn-edit { background: #f0ad4e; color: white; padding: 5px 10px; width: auto; font-size: 12px; }
    .btn-delete { background: #d9534f; color: white; padding: 5px 10px; width: auto; font-size:
12px; }
    p#msg { color: green; text-align: center; margin: 10px 0; font-weight: bold; }
    table { width: 100%; margin-top: 15px; border-collapse: collapse; }
    th { background: #1a73e8; color: white; padding: 10px; text-align: left; font-size: 13px; }
    td { border: 1px solid #ddd; padding: 8px; font-size: 13px; }
    tr:hover { background: #f5f5f5; }
    .actions { display: flex; gap: 5px; }
    hr { margin: 20px 0; border: 1px solid #eee; }
  </style>
</head>

```

```

<body>
<div class="container">
  <h2>Employee Management System</h2>
  <p style="text-align:center; color:#888; font-size:12px;">Spring Core — @Component |
@Autowired | IoC | BeanFactory</p>
  <hr>
  <div class="form-group"><label>Employee ID</label><input type="number" id="empId"
placeholder="e.g. 101"></div>
  <div class="form-group"><label>Full Name</label><input type="text" id="empName"
placeholder="e.g. Arun Kumar"></div>
  <div class="form-group"><label>Department</label><input type="text" id="empDept"
placeholder="e.g. Engineering"></div>
  <div class="form-group"><label>Salary</label><input type="number" id="empSalary"
placeholder="e.g. 75000"></div>
  <div class="btn-row">
    <button class="btn-add" id="btnSubmit" onclick="addEmployee()">Add Employee</button>
    <button class="btn-clear" onclick="clearForm()">Clear</button>
  </div>
  <p id="msg"></p>
  <hr>
  <h2>Employee Records</h2>
  <table>

<thead><tr><th>ID</th><th>Name</th><th>Department</th><th>Salary</th><th>Actions</th></tr>
</thead>
  <tbody id="employeeTableBody"></tbody>
</table>
  <p id="emptyMsg" style="text-align:center; color:#999; margin-top:10px;">No employees yet.
Add one above!</p>
</div>
<script>
  const API = '/api/employees';
  let editMode = false;
  window.onload = () => loadEmployees();
  async function loadEmployees() {
    const res = await fetch(API);
    const employees = await res.json();
    const tbody = document.getElementById('employeeTableBody');
    const emptyMsg = document.getElementById('emptyMsg');
    if (employees.length === 0) { tbody.innerHTML = ""; emptyMsg.style.display = 'block'; return; }
    emptyMsg.style.display = 'none';
    tbody.innerHTML = employees.map(emp =>
      `<tr>
        <td>${emp.id}</td>
        <td>${emp.name}</td>
        <td>${emp.department}</td>
        <td>${emp.salary}</td>
        <td class="actions">
          <button class="btn-edit"
onclick="editEmployee('${emp.id}','${emp.name}','${emp.department}','${emp.salary}')">Edit</butto
n>

```

```

        <button class="btn-delete" onclick="deleteEmployee(${emp.id})">Delete</button>
    </td>
</tr>`
).join("");
}
async function addEmployee() {
    const id = document.getElementById('empId').value;
    const name = document.getElementById('empName').value;
    const dept = document.getElementById('empDept').value;
    const salary = document.getElementById('empSalary').value;
    if (!id || !name || !dept || !salary) { alert('Please fill all fields'); return; }
    const employee = { id: parseInt(id), name, department: dept, salary: parseFloat(salary) };
    if (editMode) {
        await fetch(API + '/' + id, { method: 'PUT', headers: {'Content-Type': 'application/json'}, body:
JSON.stringify(employee) });
        document.getElementById('msg').innerText = 'Employee Updated Successfully!';
        editMode = false;
        document.getElementById('btnSubmit').innerText = 'Add Employee';
        document.getElementById('empId').disabled = false;
    } else {
        await fetch(API, { method: 'POST', headers: {'Content-Type': 'application/json'}, body:
JSON.stringify(employee) });
        document.getElementById('msg').innerText = 'Employee Added Successfully!';
    }
    clearForm(); loadEmployees();
}
function editEmployee(id, name, dept, salary) {
    editMode = true;
    document.getElementById('empId').value = id;
    document.getElementById('empName').value = name;
    document.getElementById('empDept').value = dept;
    document.getElementById('empSalary').value = salary;
    document.getElementById('empId').disabled = true;
    document.getElementById('btnSubmit').innerText = 'Update Employee';
}
async function deleteEmployee(id) {
    if (!confirm('Delete employee #' + id + '?')) return;
    await fetch(API + '/' + id, { method: 'DELETE' });
    document.getElementById('msg').innerText = 'Employee Deleted!';
    loadEmployees();
}
function clearForm() {
    document.getElementById('empId').value = "";
    document.getElementById('empName').value = "";
    document.getElementById('empDept').value = "";
    document.getElementById('empSalary').value = "";
}
</script>
</body>
</html>

```

